Практичні роботи з дисципліни «Програмна інженерія»

Проєкт: Budget & Expense Tracker — Менеджер особистих фінансів

Команда: BudgetTeam

Склад команди:

- Roman1328 — Frontend Developer.
- Yurra4 — Backend Developer.
- WizardSM — Database & Integration Engineer.

Практична робота №1

Вступ до програмної інженерії та формування команди

1. Мета роботи

Ознайомитися з основними поняттями програмної інженерії, етапами створення програмного продукту та організацією командної роботи.

2. Поняття програмної інженерії

Програмна інженерія — це галузь, яка охоплює розробку, тестування, впровадження та супровід програмного забезпечення. Вона допомагає створювати якісні програмні продукти відповідно до потреб користувачів.

Основними етапами створення програмного продукту є аналіз вимог, проєктування, розробка, тестування, впровадження та супровід.

3. Формування команди

Назва команди — **BudgetTeam**.

Учасник	**Роль**	**Обов'язки**
Roman1328	Frontend Developer	Розробка інтерфейсу, форм транзакцій, таблиць та діаграм
Yurra4	Backend Developer	Розробка REST API, авторизації та серверної логіки
WizardSM	Database & Integration Engineer	Проєктування бази даних, міграції та інтеграція API

4. Тема проєкту

Budget & Expense Tracker — Менеджер особистих фінансів.

Проєкт являє собою вебдодаток для обліку особистих доходів і витрат, перегляду балансу та аналізу фінансової статистики.

5. Проблема проєкту

Людям часто складно контролювати власні фінанси через відсутність систематизованого обліку доходів і витрат. Через це важко визначити, на що витрачається найбільше коштів, та оцінити поточний фінансовий стан.

6. Цільові користувачі

Цільовою аудиторією є студенти, працюючі люди та інші користувачі, які хочуть контролювати особисті фінанси, вести облік операцій і планувати витрати.

7. Project Vision

Бачення продукту: створити зручний вебдодаток, який допоможе користувачам вести облік доходів і витрат, контролювати баланс та аналізувати власні фінанси за допомогою статистики й діаграм.

Основні можливості продукту:

- реєстрація та вхід у систему;
- додавання, редагування та видалення транзакцій;
- перегляд поточного балансу та історії операцій;
- фільтрація за датою і категорією;
- перегляд фінансової статистики та діаграм.

Висновок: сформовано команду, визначено тему, проблему, цільову аудиторію та бачення майбутнього програмного продукту.

Практична робота №2

Аналіз проблеми, користувачів та предметної області

1. Мета роботи

Проаналізувати проблему, яку вирішує програмний продукт, визначити потреби користувачів та основні об’єкти предметної області.

2. Опис проблеми

Користувачі не завжди можуть ефективно контролювати свої доходи та витрати. Записи в різних місцях ускладнюють пошук інформації, підрахунок балансу та аналіз витрат за категоріями.

Budget & Expense Tracker має об’єднати фінансові операції в одному місці й автоматизувати підрахунок основних показників.

3. Потреби користувачів

Основні потреби:

- швидко додавати доходи та витрати;
- бачити актуальний баланс;
- переглядати історію транзакцій;
- знаходити операції за датою та категорією;
- аналізувати доходи й витрати за місяць;
- переглядати структуру витрат у вигляді діаграм.

4. Цільова аудиторія

Основними користувачами є люди, які хочуть вести особистий бюджет. Зокрема, студенти, працівники та користувачі, які прагнуть краще розуміти структуру власних витрат.

5. Основні об’єкти предметної області

Об’єкт	Опис
Користувач	Людина, яка має обліковий запис у системі
Транзакція	Фінансова операція — дохід або витрата
Категорія	Група для класифікації фінансових операцій
Баланс	Різниця між сумою доходів і витрат з урахуванням початкового залишку
Статистика	Узагальнені показники фінансових операцій
Діаграма	Графічне відображення фінансових даних

6. Взаємозв’язки між об’єктами

Один користувач може мати багато транзакцій. Кожна транзакція належить певному користувачеві та має тип, суму, дату й категорію. На основі транзакцій система обчислює баланс і формує статистику.

Категорії використовуються для групування операцій, а діаграми відображають результати аналізу.

7. Основні сценарії використання

1. Користувач реєструється та входить у систему.
2. Додає нову фінансову транзакцію.
3. Переглядає та редагує історію операцій.
4. Фільтрує транзакції за датою або категорією.
5. Переглядає баланс і статистику доходів та витрат.

8. Межі майбутньої системи

До системи входять:

- керування обліковим записом;
- облік доходів і витрат;
- керування транзакціями;
- категоризація операцій;
- розрахунок балансу;
- перегляд статистики та діаграм.

До меж першої версії не входять автоматичне підключення банківських рахунків, проведення банківських платежів, кредитування та інвестиційні операції.

Висновок: визначено проблему, потреби користувачів, основні об’єкти предметної області, їхні взаємозв’язки та межі системи.

Практична робота №3

Життєвий цикл програмного забезпечення

1. Мета роботи

Ознайомитися з моделями життєвого циклу програмного забезпечення та обрати модель, яка найкраще відповідає особливостям проєкту Budget & Expense Tracker.

2. Основні моделі життєвого циклу

Каскадна модель (Waterfall). Розробка відбувається послідовно: кожен етап завершується перед початком наступного. Підходить для проєктів зі стабільними вимогами.

Ітераційна модель. Продукт розробляється поступово, невеликими циклами. Після кожної ітерації можна перевірити результат і вдосконалити систему.

Agile. Гнучкий підхід, що передбачає короткі цикли розробки, регулярний зворотний зв'язок і можливість змінювати вимоги.

3. Порівняння моделей

Критерій	**Waterfall**	**Ітераційна**	**Agile**
Послідовність	Послідовна	Повторювані цикли	Гнучкі цикли
Зміна вимог	Ускладнена	Можлива	Передбачена підходом
Зворотний зв'язок	Переважно наприкінці	Після ітерацій	Регулярний
Зручність для навчального проєкту	Середня	Висока	Висока

4. Обрана модель

Для Budget & Expense Tracker обрано **ітераційну модель розробки**.

Вона дозволяє створювати продукт поступово, перевіряти кожну реалізовану частину та виправляти недоліки до завершення всього проєкту.

5. Етапи життєвого циклу

1. Аналіз вимог — визначення функцій і потреб користувачів.
2. Проєктування — розробка структури системи, інтерфейсу та бази даних.
3. Реалізація — створення frontend, backend та інтеграція бази даних.
4. Тестування — перевірка правильності роботи функцій.
5. Впровадження — підготовка та розгортання готового продукту.
6. Супровід — виправлення помилок і вдосконалення системи.

6. Схема життєвого циклу

Аналіз вимог → Проєктування → Реалізація → Тестування → Впровадження → Супровід

Після тестування за потреби відбувається повернення до проєктування або реалізації. Цикл повторюється для вдосконалення продукту.

7. Обґрунтування вибору

Ітераційна модель підходить для навчального проєкту, оскільки команда складається з трьох учасників, а продукт має декілька окремих функціональних частин. Спочатку можна реалізувати авторизацію та облік транзакцій, а потім додати фільтрацію, статистику й діаграми.

Висновок: розглянуто основні моделі життєвого циклу, порівняно їхні особливості та обрано ітераційну модель для розробки Budget & Expense Tracker.

Практична робота №4

Формування вимог до програмного забезпечення

1. Мета роботи

Сформувати функціональні та нефункціональні вимоги до програмного продукту, визначити пріоритети та критерії приймання.

2. Користувачі системи

Основний користувач — зареєстрована людина, яка веде облік власних фінансів. Незареєстрований відвідувач може перейти до реєстрації або входу в систему.

3. User Stories

1. Як користувач, я хочу зареєструватися, щоб створити власний обліковий запис.
2. Як користувач, я хочу входити в систему, щоб отримувати доступ до своїх фінансових даних.
3. Як користувач, я хочу додавати доходи, щоб вести їх облік.
4. Як користувач, я хочу додавати витрати, щоб контролювати використання коштів.
5. Як користувач, я хочу редагувати транзакції, щоб виправляти помилки.
6. Як користувач, я хочу видаляти транзакції, щоб прибирати непотрібні записи.
7. Як користувач, я хочу фільтрувати операції за датою та категорією, щоб швидко знаходити потрібні записи.
8. Як користувач, я хочу бачити поточний баланс, щоб оцінювати свій фінансовий стан.
9. Як користувач, я хочу переглядати діаграми витрат за категоріями, щоб розуміти структуру витрат.
10. Як користувач, я хочу переглядати місячну статистику доходів і витрат, щоб аналізувати зміни власних фінансів.

4. Функціональні вимоги

- **FR-01.** Система повинна забезпечувати реєстрацію та вхід користувачів.
- **FR-02.** Система повинна дозволяти створювати, переглядати, редагувати та видаляти транзакції.
- **FR-03.** Кожна транзакція повинна містити суму, тип, категорію та дату. Опис є необов’язковим.
- **FR-04.** Система повинна розділяти транзакції на доходи та витрати.
- **FR-05.** Система повинна автоматично розраховувати баланс на основі фінансових операцій.
- **FR-06.** Система повинна забезпечувати фільтрацію транзакцій за датою та категорією.
- **FR-07.** Система повинна формувати статистику доходів і витрат за місяць.

- **FR-08.** Система повинна відображати кругову діаграму витрат за категоріями та лінійний графік доходів і витрат.
- **FR-09.** Система повинна показувати останні транзакції користувача.
- **FR-10.** Система повинна забезпечувати доступ користувача лише до його власних фінансових даних.

5. Нефункціональні вимоги

- **NFR-01. Зручність:** інтерфейс повинен бути зрозумілим і простим у використанні.
- **NFR-02. Безпека:** паролі повинні зберігатися в захищеному вигляді, а доступ до API має перевірятися.
- **NFR-03. Продуктивність:** основні операції та перегляд статистики повинні виконуватися без помітних затримок за нормального навантаження.
- **NFR-04. Надійність:** система повинна перевіряти введені дані та повідомляти про помилки.
- **NFR-05. Сумісність:** інтерфейс повинен коректно відображатися в сучасних браузерах на комп'ютерах і мобільних пристроях.
- **NFR-06. Підтримуваність:** код повинен мати зрозумілу структуру та бути розділеним на логічні модулі.

6. Пріоритети вимог

Пріоритет	Вимоги
Високий	Реєстрація, вхід, CRUD-операції з транзакціями, розрахунок балансу, захист даних
Середній	Фільтрація, категорії, місячна статистика, діаграми
Низький	Додаткові можливості налаштування відображення статистики

7. Критерії приймання основних User Stories

US-01. Реєстрація

- Користувач може створити обліковий запис, заповнивши обов'язкові поля.
- Якщо дані некоректні або обліковий запис уже існує, система показує повідомлення про помилку.
- Після успішної реєстрації користувач може увійти в систему.

US-03. Додавання транзакції

- Користувач може вказати суму, тип, категорію та дату.
- Система не приймає некоректну або непозитивну суму.
- Після збереження транзакція з'являється в історії, а баланс оновлюється.

US-08. Перегляд балансу

- Система відображає загальну суму доходів, витрат і поточний баланс.
- Баланс відповідає збереженим транзакціям.
- Після додавання, редагування або видалення операції баланс перераховується.

US-09. Перегляд діаграми

- Система групує витрати за категоріями.
- Діаграма відображає дані з транзакцій користувача.
- Після зміни операцій статистика оновлюється.

Висновок: сформовано User Stories, функціональні та нефункціональні вимоги, визначено їхні пріоритети й критерії приймання основних функцій системи.

Практична робота №5

User Stories та критерії приймання

1. Мета роботи

Навчитися описувати функціональність програмного продукту з погляду користувача та формувати критерії приймання для перевірки реалізованих функцій.

2. Структура User Story

User Story описує потребу користувача за шаблоном:

Як [роль], я хочу [дія], щоб [очікуваний результат].

Кожна історія повинна бути зрозумілою, мати конкретну мету та критерії, за якими можна перевірити її виконання.

3. Backlog User Stories

US-01. Реєстрація користувача

Як новий користувач, я хочу зареєструватися, щоб створити власний обліковий запис.

Критерії приймання:

- можна ввести необхідні реєстраційні дані;
- некоректні дані спричиняють повідомлення про помилку;
- повторна реєстрація з уже зайнятою електронною адресою не допускається.

US-02. Вхід у систему

Як зареєстрований користувач, я хочу входити в систему, щоб переглядати свої фінансові дані.

Критерії приймання:

- правильні облікові дані дозволяють увійти;
- неправильні дані спричиняють повідомлення про помилку;
- користувач отримує доступ лише до власних даних.

US-03. Додавання доходу

Як користувач, я хочу додавати доходи, щоб відстежувати надходження коштів.

Критерії приймання:

- можна вказати суму, категорію та дату;
- система перевіряє правильність введених даних;

- після збереження дохід відображається в історії та враховується в балансі.

US-04. Додавання витрати

Як користувач, я хочу додавати витрати, щоб контролювати використання коштів.

Критерії приймання:

- можна вказати суму, категорію та дату;
- сума повинна бути більшою за нуль;
- витрата відображається в історії та враховується під час розрахунку балансу.

US-05. Редагування транзакції

Як користувач, я хочу редагувати транзакцію, щоб виправляти неправильні дані.

Критерії приймання:

- можна змінити доступні поля транзакції;
- система перевіряє введені значення;
- після збереження історія та баланс відображають актуальні дані.

US-06. Видалення транзакції

Як користувач, я хочу видаляти транзакції, щоб прибирати помилкові записи.

Критерії приймання:

- користувач може вибрати власну транзакцію для видалення;
- система повідомляє про успішне видалення;
- баланс перераховується після видалення.

US-07. Фільтрація транзакцій

Як користувач, я хочу фільтрувати операції за датою та категорією, щоб швидко знаходити потрібні записи.

Критерії приймання:

- можна встановити фільтр за датою;
- можна вибрати категорію;
- результати відповідають заданим умовам фільтрації.

US-08. Перегляд балансу

Як користувач, я хочу бачити свій баланс, щоб розуміти поточний стан фінансів.

Критерії приймання:

- система показує загальний баланс;
- доходи додаються, а витрати віднімаються;
- значення оновлюється після зміни транзакцій.

US-09. Перегляд статистики за категоріями

Як користувач, я хочу переглядати діаграму витрат за категоріями, щоб визначати основні напрями використання коштів.

Критерії приймання:

- система підраховує витрати для кожної категорії;
- діаграма відповідає збереженим даним;
- категорії без витрат не створюють помилкових значень.

US-10. Місячна статистика

Як користувач, я хочу переглядати доходи та витрати за місяць, щоб аналізувати фінансові зміни.

Критерії приймання:

- система показує суму доходів за обраний місяць;
- система показує суму витрат за обраний місяць;
- дані відповідають транзакціям за відповідний період.

4. Перевірка повноти User Stories

Сформовані історії містять роль користувача, бажану дію та очікувану користь. Для кожної історії визначено критерії приймання, які дозволяють перевірити правильність реалізації.

Backlog охоплює основні можливості першої версії продукту: авторизацію, облік транзакцій, керування операціями, фільтрацію, баланс і фінансову аналітику.

Висновок: сформовано backlog із десяти User Stories та критеріїв приймання, що можуть використовуватися командою під час розробки й тестування Budget & Expense Tracker.

Практична робота №6

Аналіз предметної області та Use Case Diagram

1. Мета роботи

Визначити акторів і сценарії взаємодії з програмним продуктом, побудувати Use Case Diagram та описати основні сценарії використання системи.

2. Актори системи

1. Відвідувач. Людина, яка ще не має активного облікового запису. Може зареєструватися або перейти до входу.

2. Зареєстрований користувач. Людина, яка має доступ до власного облікового запису та може керувати фінансовими операціями й переглядати статистику.

3. Основні сценарії використання

Актор	**Сценарії**
Відвідувач	Реєстрація, вхід у систему
Зареєстрований користувач	Додавання доходу, додавання витрати, перегляд транзакцій, редагування, видалення, фільтрація, перегляд балансу, перегляд статистики

4. Use Case Diagram

Діаграма відображає взаємодію двох акторів із системою Budget & Expense Tracker.

Для побудови діаграми можна використати PlantUML:

```
@startuml
left to right direction

actor "Відвідувач" as Guest
actor "Користувач" as User

rectangle "Budget & Expense Tracker" {
  usecase "Реєстрація" as UC1
  usecase "Вхід у систему" as UC2
  usecase "Додати дохід" as UC3
  usecase "Додати витрату" as UC4
  usecase "Переглянути транзакції" as UC5
  usecase "Редагувати транзакцію" as UC6
  usecase "Видалити транзакцію" as UC7
  usecase "Фільтрувати транзакції" as UC8
  usecase "Переглянути баланс" as UC9
  usecase "Переглянути статистику" as UC10
}

Guest --> UC1
Guest --> UC2

User --> UC2
User --> UC3
User --> UC4
User --> UC5
User --> UC6
User --> UC7
User --> UC8
User --> UC9
User --> UC10
```

@enduml

5. Опис основних Use Cases

Use Case 1. Додавання фінансової транзакції

Актор: зареєстрований користувач.

Мета: зберегти нову фінансову операцію.

Передумова: користувач увійшов у систему.

Основний сценарій:

1. Користувач відкриває форму додавання транзакції.
2. Обирає тип операції — дохід або витрата.
3. Вводить суму, категорію, дату та, за потреби, опис.
4. Підтверджує збереження.
5. Система перевіряє введені дані.
6. Система зберігає транзакцію та оновлює баланс.

Альтернативний сценарій: якщо дані некоректні, система показує повідомлення про помилку та пропонує виправити введені значення.

Результат: транзакцію збережено, баланс оновлено.

Use Case 2. Перегляд фінансової статистики

Актор: зареєстрований користувач.

Мета: отримати інформацію про доходи та витрати.

Передумова: користувач увійшов у систему та має доступ до фінансової панелі.

Основний сценарій:

1. Користувач відкриває сторінку статистики.
2. Система отримує фінансові дані користувача.
3. Обчислюються доходи та витрати за місяць.
4. Система групує витрати за категоріями.
5. На сторінці відображаються показники й діаграми.

Альтернативний сценарій: якщо транзакцій немає, система показує нульові показники або повідомлення про відсутність даних.

Результат: користувач бачить актуальну фінансову статистику.

Use Case 3. Редагування транзакції

Актор: зареєстрований користувач.

Мета: виправити або оновити дані фінансової операції.

Передумова: користувач увійшов у систему та має власну збережену транзакцію.

Основний сценарій:

1. Користувач відкриває історію транзакцій.
2. Обирає потрібну операцію.
3. Змінює суму, тип, категорію, дату або опис.
4. Підтверджує зміни.
5. Система перевіряє та зберігає нові значення.
6. Баланс і статистика оновлюються відповідно до змін.

Альтернативний сценарій: якщо введені дані некоректні, система повідомляє про помилку та не зберігає некоректні зміни.

Результат: транзакцію оновлено, фінансові показники перераховано.

6. Аналіз взаємодії

Відвідувач взаємодіє із системою через реєстрацію та вхід. Після авторизації користувач отримує доступ до функцій обліку фінансів. Додавання, редагування та видалення транзакцій впливають на баланс і статистику. Фільтрація допомагає знаходити необхідні операції.

Висновок: визначено акторів системи, основні сценарії використання та зв’язки між ними. Створено опис Use Case Diagram і трьох основних сценаріїв взаємодії користувача з Budget & Expense Tracker.